

Smart Vehicle Service Management System

1. Objective of the Proposal

Manual vehicle service tracking is slow and error-prone.
Develop a web-based system using Python Flask and MySQL.
Database ensures every service is linked to vehicle, owner, and parts.

2. Project Team Information

- **Student 1 Name:** Asim Ali, BSCS "A"
 - **Student 2 Name:** Tariq Jamil, BSCS "A"
 - **Project Title:** Smart Vehicle Service Management System
-

3. Introduction & Background

Garages often use paper or spreadsheets to track services.
Records are frequently lost or inconsistent.
Inventory management is difficult without a system.
A database-backed system ensures fast access and accurate records.
Improves customer service and operational efficiency.

4. Problem Statement

Garages have "Information Silos."
Mechanics don't know vehicle history.
Owners forget last service dates.
Spare parts run out due to no automated alerts.
A database system provides persistence and fast retrieval.

5. Proposed Solution

Dashboard to register customers and vehicles.
Log services and assign mechanics.
Deduct parts automatically from inventory.
Generate reports on daily earnings and stock levels.

6. Objectives of the System

Efficient management of vehicles, owners, and services.
Maintain data consistency and integrity.
Fast retrieval and updates of records.
Accurate spare parts tracking.
Generate service and inventory reports.

7. Scope of the Project

Included:

User login (Admin/Staff)
Vehicle and owner profiles
Service logging
Inventory management (CRUD)
Basic reports (PDF/On-screen)

Excluded:

Online payments
SMS notifications
AI-based diagnostics

8. Role of Database System

Data Stored: Vehicles, owners, services, spare parts, mechanics.
Need: Centralized storage, fast access, accurate reporting.
Operations: CRUD, search, reporting.
Integrity: Primary keys, foreign keys, and constraints prevent errors.

9. System Features / Functional Requirements

Role-based access: Admin vs Staff
View full service history per plate number
Low stock alerts for spare parts
Search/filter by owner, plate, or date
Add/update/delete vehicles, owners, and services
Inventory management with automatic updates
Generate reports: service, inventory, earnings
Dashboard for overview

10. Preliminary Data Design

Entities & Tables:

Owner: OwnerID, Name, Phone, Address

Vehicle: VehicleID, PlateNumber, Model, Year, OwnerID

Service: ServiceID, VehicleID, ServiceDate, TotalCost, MechanicID

Inventory: PartID, PartName, StockQuantity, UnitPrice

Service_Parts: ServiceID, PartID, QuantityUsed

Mechanic: MechanicID, Name, Specialization

Relationships:

Owner → Vehicle (1:M)

Vehicle → Service (1:M)

Service → Mechanic (M:1)

Service ↔ Inventory (M:N via Service_Parts)

11. Existing Systems / Comparative Analysis

Existing: Paper records, Excel sheets, basic garage software.

Limitations: Slow, error-prone, poor reporting.

Improvement: Relational DB enables faster search, inventory management, and reports.

12. Proposed Technology Stack

Frontend: HTML, CSS (Bootstrap), JavaScript

Backend: Python Flask

Database: MySQL

Optional ORM: SQLAlchemy

13. Expected Outcomes

No duplicate vehicle or owner records.

Fast retrieval of old service records.

Real-time inventory updates.

Efficient service and parts management.

Accurate management reports.